

Deep Learning and Its Application

Week 6: Practicals

1. Given one input sample with 3 time steps $t_1 = 0.1$, $t_2 = 0.2$, $t_3 = 0.3$ and one feature observed at each time step. Make the following code working and return a sequence of 3 values, one hidden state output for each input time step for the single LSTM cell in the layer.

```
from keras import Model
from keras.layers import Input
from keras.layers import LSTM
from numpy import array
# define model
inputs1 = Input(shape=(3, 1))
#output a single sequence
#lstm1 = LSTM(1)(inputs1)
#output three sequences
lstm1 = LSTM(1)(inputs1)
model = Model(inputs=inputs1, outputs=lstm1)
# define input data
data = array([0.1, 0.2, 0.3]).reshape((1,3,1))
# make and show prediction
print(model.predict(data))
```

2. Given one input sample with 3 time steps and one feature observed at each time step: $t_1 = 0.1$, $t_2 = 0.2$, $t_3 = 0.3$. Make the following code working and return:
 - a. A LSTM hidden state output for the last time step.
 - b. A LSTM hidden state output for the last time step (again).
 - c. A LSTM cell state for the last time step.

```
from keras.models import Model
from keras.layers import Input
from keras.layers import LSTM
from numpy import array
# define model
inputs1 = Input(shape=(3, 1))
lstm1, state_h, state_c = LSTM(1, )(inputs1)
model = Model(inputs=inputs1, outputs=[lstm1, state_h, state_c])
# define input data
data = array([0.1, 0.2, 0.3]).reshape((1,3,1))
# make and show prediction
print(model.predict(data))
```

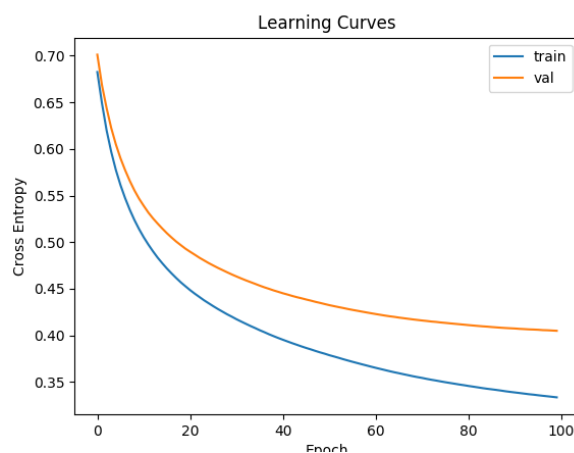
3. Given the following fragment code

```
from sklearn.datasets import make_classification
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
#from matplotlib import pyplot

import matplotlib
matplotlib.use('TkAgg')
import matplotlib.pyplot as plt
```

```
# create the dataset
X, y = make_classification(n_samples=1000, n_classes=2, random_state=1)
# determine the number of input features
n_features = X.shape[1]
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal',
input_shape=(n_features,)))
model.add(Dense(1, activation='sigmoid'))
# compile the model
sgd = SGD(learning_rate=0.001, momentum=0.8)
model.compile(optimizer=sgd, loss='binary_crossentropy')
# fit the model
history = model.fit(X, y, epochs=100, batch_size=32, verbose=0,
validation_split=0.3)
```

extend the above code and plot the following diagram

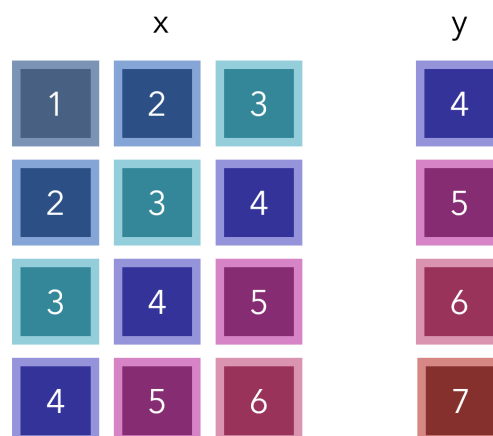


Tips: history = model.fit(), then 'history' contains 'loss' (train) and 'val_loss' (val)

4. Consider the following sequence



and generate the following set of sequences, in this case the window size is 3



Use the Sequential model and “SimpleRNN” to develop a stack RNN with the following tasks

- a) Construct 4 hidden layer with “SimpleRNN”, the respective layers consist of 20, 32, 16, 8 neurons
 - b) Add “return_sequences=True”
 - c) Add “tanh” activation function in one layer
 - d) The output layer with “Linear” activation function
 - e) Add optimizer with “Adm”, loss function with “mse”, and metrics with “mae”
 - f) Run your model with epochs = 10, 50, 500
 - g) Evaluate the model with the training set of sequences
 - h) Given an input sequence [2, 3, 4], make predictions
5. Use LSTM to develop an algorithm for predicting the number of car sales per month. The dataset will be downloaded automatically using Pandas, you can learn more about the dataset in Appendix.

Formulate the problem to take a window of the last five months of data to predict the current month’s data.

To achieve this, a new function named split_sequence() is provided below that will split the input sequence into windows of data appropriate for fitting a LSTM model.

```
from numpy import sqrt
from numpy import asarray
from pandas import read_csv
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import LSTM

# split a univariate sequence into samples
def split_sequence(sequence, n_steps):
    X, y = list(), list()
    for i in range(len(sequence)):
        # find the end of this pattern
        end_ix = i + n_steps
        # check if we are beyond the sequence
        if end_ix > len(sequence)-1:
            break
        # gather input and output parts of the pattern
        seq_x, seq_y = sequence[i:end_ix], sequence[end_ix]
        X.append(seq_x)
        y.append(seq_y)
    return asarray(X), asarray(y)

# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/monthly-car-sales.csv'
df = read_csv(path, header=0, index_col=0, squeeze=True)
# retrieve the values
values = df.values.astype('float32')
# specify the window size
n_steps = 5
# split into samples
X, y = split_sequence(values, n_steps)
# reshape into [samples, timesteps, features]
```

```

X = X.reshape((X.shape[0], X.shape[1], 1))
# split into train/test
n_test = 12
X_train, X_test, y_train, y_test = X[:-n_test], X[-n_test:], y[:-n_test], y[-n_test:]
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# define model
model = Sequential()
.
.
.
print('MSE: %.3f, RMSE: %.3f, MAE: %.3f' % (mse, sqrt(mse), mae))
# make a prediction
row = asarray([18024.0, 16722.0, 14385.0, 21342.0, 17180.0]).reshape((1, n_steps, 1))
yhat = model.predict(row)
print('Predicted: %.3f' % (yhat))

```

Use the Sequential model and “LSTM ” to develop a LSTM algorithm for predicting the number of car sales based on input [18024.0, 16722.0, 14385.0, 21342.0, 17180.0].

Appendix

Car Sales Dataset Description

Monthly car sales in Quebec 1960-1968.

Source: Time Series Data Library (citing: Abraham & Ledolter (1983))

Car Sales Dataset

```

"Month","Sales"
"1960-01",6550
"1960-02",8728
"1960-03",12026
"1960-04",14395
"1960-05",14587
"1960-06",13791
"1960-07",9498
"1960-08",8251
"1960-09",7049
"1960-10",9545
"1960-11",9364
"1960-12",8456
"1961-01",7237
"1961-02",9374
"1961-03",11837
"1961-04",13784
"1961-05",15926
"1961-06",13821
"1961-07",11143
.

```

